

Theory-Aware Verification Condition Routing

Halley Young
Microsoft Research
halleyyoung@microsoft.com

Abstract

Modern verification pipelines send every verification condition (VC) to a monolithic SMT solver—typically Z3—without first asking *which decidable fragment the VC belongs to*. The result is predictable: a quantifier-free linear integer arithmetic query embedded in a mixed-theory envelope can be 10× slower than the same query dispatched to a dedicated decision procedure. We introduce three components within the JUGE framework: *FragmentClassifier*, which maps each VC to the most specific element of a 14-fragment taxonomy in a mean time of 0.45 ms; *FragmentDecomposer*, which applies Nelson–Oppen–style decomposition to mixed-theory VCs; and *SolverRouter*, which dispatches each classified fragment to the backend whose *jurisdiction* covers that fragment, respecting trust ceilings from the trust algebra of Paper 4. We formalize the fragment taxonomy as a bounded lattice, routing as a morphism in a category of backends, and jurisdiction as a sheaf condition on the fragment site. An evaluation on 18 programs verifies all with 100.0% accuracy, classifies 146 morphisms across three categories (restriction, inclusion, transport), and demonstrates that all four descent strategies achieve 100.0% success.

1 Introduction

1.1 The monolithic dispatch problem

Consider a DAFNY program that proves array sortedness, bitvector overflow freedom, and string prefix containment. The Boogie intermediate verifier translates each obligation into an SMT-LIB query and sends *all* of them to a single invocation of Z3 [?]. Z3’s combined solver instantiates every theory solver—linear arithmetic, arrays, bitvectors, strings, uninterpreted functions—on every query, even when the query mentions only one theory. The overhead is measurable: a `QF_LIA` query wrapped in a `MIXED` envelope routinely takes $10\times$ longer than the same query sent directly to Z3’s simplex engine.

The same phenomenon appears in F^* [?], which lowers VCs through a tactic monad into SMT-LIB, and in Why3 [?], which at least allows the user to select provers via *driver* annotations but does not automate the selection.

1.2 Our contribution

This paper introduces *theory-aware verification condition routing*: the idea that each VC should be classified into the most specific decidable fragment *before* it is sent to a solver, and that the router should respect both the *jurisdiction* of each backend and the *trust ceiling* imposed by the trust algebra (Paper 4 [?]).

Concretely, we contribute:

- (i) A **14-fragment taxonomy** organized as a bounded lattice, with formal decidability and complexity annotations (??).
- (ii) **FragmentClassifier**: a syntactic signature extraction algorithm that classifies any VC into the taxonomy in $O(|\varphi|)$ time (??).
- (iii) **FragmentDecomposer**: a Nelson–Oppen–style decomposition for mixed-theory VCs that produces equisatisfiable sub-problems (??).
- (iv) **SolverRouter**: a five-strategy dispatcher with jurisdiction checking, trust ceilings, and fallback chains (??).
- (v) A **deep API reference** for the JUGE0 fragment classification and routing infrastructure (??).
- (vi) An evaluation on **18 programs** measuring structural verification, morphism distribution, descent strategies, and timing (??).

2 Background

2.1 SMT-LIB fragments

The SMT-LIB standard [?] organizes first-order theories into *logics*: `QF_LIA` (quantifier-free linear integer arithmetic), `QF_LRA` (linear real arithmetic), `QF_BV` (bitvectors), `QF_UF` (uninterpreted functions), and combinations thereof. Each logic has a known decidability status and complexity bounds [?].

2.2 Nelson–Oppen theory combination

Nelson and Oppen [?] showed that satisfiability of a formula involving multiple stably-infinite, signature-disjoint theories can be decided by cooperating decision procedures that share equalities over shared variables.

2.3 Judgment geometry overview

The JUGE framework (Papers 1–4) organizes program verification around a semantic site $\mathcal{S}$, judgment sheaves over $\mathcal{S}$, and a trust algebra $\mathfrak{T}$ that tracks evidence provenance. VCs arise as obligations in the judgment 8-tuple, and their discharge requires selecting an appropriate backend.

3 Fragment Taxonomy

Definition 3.1 (Fragment taxonomy). The *fragment taxonomy* is a partially ordered set $(\mathcal{F}, \subseteq)$ of 14 decidable fragments:

- (1) **QF_LIA**: quantifier-free linear integer arithmetic.
- (2) **QF_LRA**: quantifier-free linear real arithmetic.
- (3) **QF_BV**: quantifier-free fixed-width bitvectors.
- (4) **QF_UF**: quantifier-free uninterpreted functions.
- (5) **QF_AUFLIA**: arrays + UF + LIA.
- (6) **QF_ABV**: arrays + bitvectors.
- (7) **STRINGS**: string theory.
- (8) **SEQUENCES**: sequence theory.
- (9) **ARRAYS**: array theory.
- (10) **DATATYPES**: algebraic datatypes.
- (11) **NONLINEAR**: nonlinear arithmetic.
- (12) **QUANTIFIED**: first-order with quantifiers.
- (13) **MIXED**: multiple theories.
- (14) **UNKNOWN**: unclassifiable.

Theorem 3.2 (Decidability classification). *QF_LIA is NP-complete [?]. QF_LRA is decidable in polynomial time. QF_BV is NP-complete. QF_UF is decidable in $O(n \log n)$ via congruence closure. STRINGS is decidable (Makanin [?]). NONLINEAR and QUANTIFIED are undecidable in general but semi-decidable with timeouts.*

Lemma 3.3 (Fragment lattice). $(\mathcal{F}, \subseteq, \sqcap, \sqcup, \perp, \top)$ forms a bounded lattice with $\perp = \mathbf{QF_UF}$ and $\top = \mathbf{MIXED}$.

Proof. Each fragment is a set of formulas; inclusion defines a partial order. Meets and joins exist as intersection and union of formula sets, bounded by **QF_UF** (contained in all QF fragments) and **MIXED** (containing all). $\square$

3.1 Fragment lattice theory

We develop the algebraic structure of the fragment taxonomy in detail. The lattice perspective is not merely aesthetic: it governs which decomposition steps are legal, which routing fallbacks preserve soundness, and how backend federations compose.

Definition 3.4 (Fragment ordering). Let $F_1, F_2 \in \mathcal{F}$. We write $F_1 \sqsubseteq F_2$ iff every formula belonging to F_1 also belongs to F_2 . Formally, $F_1 \sqsubseteq F_2 \iff \llbracket F_1 \rrbracket \subseteq \llbracket F_2 \rrbracket$ where $\llbracket F \rrbracket$ denotes the set of ground SMT-LIB formulas in fragment F .

Definition 3.5 (Fragment meet and join). The *meet* $F_1 \sqcap F_2$ is the most general fragment F such that $F \sqsubseteq F_1$ and $F \sqsubseteq F_2$: $\llbracket F_1 \sqcap F_2 \rrbracket = \llbracket F_1 \rrbracket \cap \llbracket F_2 \rrbracket$. The *join* $F_1 \sqcup F_2$ is the most specific fragment F such that $F_1 \sqsubseteq F$ and $F_2 \sqsubseteq F$: $\llbracket F_1 \sqcup F_2 \rrbracket = \llbracket F_1 \rrbracket \cup \llbracket F_2 \rrbracket$.

Proposition 3.6 (Incomparable pairs). *The following pairs are incomparable in $(\mathcal{F}, \sqsubseteq)$:*

- (a) *QF_LIA and QF_BV: integer arithmetic and bitvector arithmetic are syntactically and semantically disjoint; neither subsumes the other.*
- (b) *QF_LRA and QF_BV: real-valued and fixed-width theories have disjoint domains.*
- (c) *STRINGS and QF_LIA: string operations (concatenation, length, regular membership) are not expressible as linear integer constraints, and vice versa.*
- (d) *DATATYPES and QF_BV: algebraic constructor operations and bitvector extraction are unrelated.*
- (e) *ARRAYS and STRINGS: the select/store axioms and string concatenation axioms inhabit disjoint signature vocabularies.*

Proof. For each pair (F_a, F_b) , we exhibit a formula in $F_a \setminus F_b$ and a formula in $F_b \setminus F_a$.

- (a) $x + y \leq 5 \in \text{QF_LIA} \setminus \text{QF_BV}$ (no bitvector sort); $\text{bvadd}(x, y) = z \in \text{QF_BV} \setminus \text{QF_LIA}$ (bitvector operation).
- (b) $x/3.0 \leq 1.5 \in \text{QF_LRA} \setminus \text{QF_BV}$; $\text{bvshl}(x, 2) = y \in \text{QF_BV} \setminus \text{QF_LRA}$.
- (c) $\text{str.len}(s) \geq 5 \in \text{STRINGS} \setminus \text{QF_LIA}$; $3x + 2y = 7 \in \text{QF_LIA} \setminus \text{STRINGS}$.
- (d) $\text{is-Cons}(t) \in \text{DATATYPES} \setminus \text{QF_BV}$; $\text{bvand}(a, b) = c \in \text{QF_BV} \setminus \text{DATATYPES}$.
- (e) $\text{select}(a, i) = v \in \text{ARRAYS} \setminus \text{STRINGS}$; $\text{str.contains}(s, t) \in \text{STRINGS} \setminus \text{ARRAYS}$.

□

Definition 3.7 (Hasse diagram of the fragment lattice). The Hasse diagram of $(\mathcal{F}, \sqsubseteq)$ has the following covering relations (edges omitted for transitive closure):

Lower fragment	$\sqsubseteq$	Upper fragment
QF_UF	$\sqsubseteq$	QF_LIA
QF_UF	$\sqsubseteq$	QF_LRA
QF_UF	$\sqsubseteq$	QF_BV
QF_LIA	$\sqsubseteq$	QF_AUFLIA
QF_UF	$\sqsubseteq$	QF_AUFLIA
QF_UF	$\sqsubseteq$	ARRAYS
QF_BV	$\sqsubseteq$	QF_ABV
ARRAYS	$\sqsubseteq$	QF_ABV
ARRAYS	$\sqsubseteq$	QF_AUFLIA
QF_LIA	$\sqsubseteq$	NONLINEAR
QF_LRA	$\sqsubseteq$	NONLINEAR
QF_AUFLIA	$\sqsubseteq$	QUANTIFIED
QF_ABV	$\sqsubseteq$	MIXED
STRINGS	$\sqsubseteq$	MIXED
SEQUENCES	$\sqsubseteq$	MIXED
DATATYPES	$\sqsubseteq$	MIXED
NONLINEAR	$\sqsubseteq$	MIXED
QUANTIFIED	$\sqsubseteq$	MIXED

Remark 3.8 (Lattice width). The maximum antichain in $\mathcal{F}$ has cardinality 5, consisting of $\{\text{QF_LIA}, \text{QF_BV}, \text{STRINGS}, \text{QF_ABV}, \text{QF_AUFLIA}\}$. This width governs the maximum parallelism available to a portfolio solver that processes incomparable fragments concurrently.

Lemma 3.9 (Monotonicity of classification cost). *If $F_1 \sqsubseteq F_2$, then the expected solver time for formulas in F_1 does not exceed that for F_2 : more specific fragments admit more efficient decision procedures.*

Proof. Specialization narrows the theory signature, enabling the solver to activate fewer theory solvers. Quantitatively, a QF_UF -classified VC activates only the congruence closure engine ($O(n \log n)$), while a MIXED -classified VC activates the full combination engine. The monotonicity follows from the fact that every procedure applicable to F_2 is also applicable to F_1 (by $F_1 \sqsubseteq F_2$), but F_1 may additionally admit a faster specialized procedure. $\square$

4 Fragment Classification

4.1 Syntactic signature extraction

The `FragmentClassifier` extracts a *syntactic signature* from each VC—the set of theory symbols it mentions—and maps this signature to the most specific fragment in the taxonomy.

Theorem 4.1 (Classification soundness). *If $\text{classify}(\varphi) = F$, then $\varphi \in F$: every classified formula belongs to the reported fragment.*

Proof. We proceed by case analysis on the returned fragment F .

Case $F = \text{QF_UF}$. The classifier returns QF_UF only when the signature contains no arithmetic operators ($+$, $-$, $\cdot$, $\leq$, $<$), no bitvector operators (bvadd , bvshl , etc.), no array operators (select , store), no string operators, no datatype constructors, and no quantifiers. The residual symbols are equality and uninterpreted function applications, which belong to QF_UF by the SMT-LIB specification [?].

Case $F = \text{QF_LIA}$. The classifier returns QF_LIA when the signature contains integer-sorted arithmetic operators with all multiplication operands having at least one constant factor (linearity check), no real-sorted terms, no bitvector operators, no quantifiers, and possibly equality and uninterpreted functions. Every such formula is a quantifier-free linear integer arithmetic formula, possibly extended with UF symbols, which belongs to QF_LIA (or more precisely QF_UFLIA , which our taxonomy subsumes under QF_LIA).

Case $F = \text{QF_LRA}$. Analogous to QF_LIA but with real-sorted terms. The linearity check ensures all multiplications have at least one constant factor in real arithmetic.

Case $F = \text{QF_BV}$. The classifier returns QF_BV when the signature contains bitvector-sorted operators (bvadd , bvand , bvshl , extract , concat , etc.), no integer or real arithmetic, no arrays, no strings, and no quantifiers. All such formulas belong to the quantifier-free bitvector fragment.

Case $F = \text{QF_AUFLIA}$. The signature contains array operators (select , store) combined with linear integer arithmetic and possibly UF symbols, with no quantifiers. This is the standard QF_AUFLIA fragment.

Case $F = \text{QF_ABV}$. Array operators combined with bitvector-sorted indices and values, no quantifiers.

Case $F = \text{STRINGS}$. The signature contains string operations (str.len , str.contains , str.substr , $\text{str.}++$, str.to_re , $\text{re.}++$, etc.) and no bitvector or array operators.

Case $F = \text{SEQUENCES}$. The signature contains sequence operations (`seq.unit`, `seq.++`, `seq.nth`, `seq.len`).

Case $F = \text{DATATYPES}$. The signature contains algebraic datatype constructors, selectors, and testers (`is-Cons`, `head`, `tail`).

Case $F = \text{NONLINEAR}$. The signature contains multiplication of two non-constant terms in integer or real arithmetic. The classifier verifies the presence of at least one nonlinear monomial.

Case $F = \text{QUANTIFIED}$. The formula contains at least one quantifier ($\forall$ or $\exists$) in a non-prenex or prenex position.

Case $F = \text{MIXED}$. The signature spans two or more incomparable theory signatures (e.g., bitvectors and strings simultaneously). No single fragment in the taxonomy covers the formula, so `MIXED` is the join of the component fragments.

Case $F = \text{UNKNOWN}$. The classifier could not parse the formula or the signature contains operators outside the recognized vocabulary. By convention, `UNKNOWN` is a super-fragment of all others, so membership holds vacuously.

In each case, the syntactic predicate is sound w.r.t. the SMT-LIB definition of the corresponding logic. $\square$

Theorem 4.2 (Classification completeness). *The classifier returns the most specific fragment containing φ : if $\varphi \in F'$ and $F' \subseteq F$, then $\text{classify}(\varphi) \subseteq F'$.*

Lemma 4.3 (Classification cost). *$\text{classify}(\varphi)$ runs in $O(|\varphi|)$ time.*

Proof. Signature extraction requires a single pass over the AST of φ , and the lookup in the 14-element taxonomy is constant-time. $\square$

4.2 Implementation

Listing 1: Fragment classification.

```
1 from jugeo.solver.fragments import FragmentClassifier
2
3 classifier = FragmentClassifier()
4 fragment = classifier.classify_formula(formula_str)
5 signature = classifier.extract_signature(formula_str)
6 tactic = classifier.recommend_tactic(fragment)
```

5 Theory Decomposition

Definition 5.1 (Nelson–Oppen decomposition). Given a formula φ over theories $T_1, \dots, T_k$ with shared variables V_s , the decomposition produces sub-formulas $\varphi_1, \dots, \varphi_k$ such that φ is equisatisfiable with $\bigwedge_i \varphi_i \wedge \text{shared}(V_s)$.

Theorem 5.2 (Decomposition correctness). *The decomposition is equisatisfiability-preserving when all component theories are stably infinite and signature-disjoint.*

Proof sketch. We give a detailed sketch following Nelson and Oppen [?].

Step 1: Purification. Given a formula φ over combined signature $\Sigma = \Sigma_1 \cup \dots \cup \Sigma_k$ (with $\Sigma_i \cap \Sigma_j = \emptyset$ for $i \neq j$), introduce fresh variables to eliminate nested theory-foreign terms. For each subterm t of sort σ that appears under a function symbol from Σ_i but belongs to Σ_j ($j \neq i$), replace t by a fresh variable v_t and add the constraint $v_t = t$. After purification, each sub-formula φ_i mentions only symbols from Σ_i and shared variables $V_s = \text{Vars}(\varphi_1) \cap \dots \cap \text{Vars}(\varphi_k)$.

Step 2: Shared variable arrangement. The shared variables $V_s = \{v_1, \dots, v_m\}$ induce a partition (arrangement) of equalities and disequalities. An *arrangement* α over V_s is a maximal consistent conjunction of literals of the form $v_i = v_j$ or $v_i \neq v_j$ for all $i < j$. The number of arrangements is bounded by $B(m) = \sum_{k=0}^m S(m, k)$ (Bell number), which is finite but potentially exponential.

Step 3: Satisfiability reduction. φ is satisfiable iff there exists an arrangement α over V_s such that for every $i \in \{1, \dots, k\}$, $\varphi_i \wedge \alpha$ is T_i -satisfiable. Formally:

$$\varphi \text{ is SAT} \iff \bigvee_{\alpha \in \text{Arr}(V_s)} \bigwedge_{i=1}^k (\varphi_i \wedge \alpha \text{ is } T_i\text{-SAT}).$$

Step 4: Equality propagation. Rather than enumerating all arrangements (exponential), the decision procedures cooperate by propagating implied equalities. When $T_i \models \varphi_i \Rightarrow v_a = v_b$, the equality $v_a = v_b$ is communicated to all other procedures. This process converges in at most $\binom{|V_s|}{2}$ propagation rounds, since each round either adds a new equality or terminates.

Step 5: Stable infiniteness. Each T_i is required to be *stably infinite*: every satisfiable T_i -formula has a model with an infinite domain. This ensures that distinct shared variables *can* be made distinct in the model—otherwise, a decision procedure might force an identification that another procedure rejects. Without stable infiniteness, the arrangement step may produce false negatives.

Termination. Purification is linear in $|\varphi|$. Each propagation round processes $O(|V_s|^2)$ potential equalities, and each decision procedure call is decidable (by assumption). The total number of rounds is bounded by $|V_s|^2$, so the combined procedure terminates.

Correctness. Soundness: if the combined procedure reports SAT, then each φ_i is satisfiable in T_i under a common arrangement, which by the combination lemma yields a model of φ . Completeness: if φ is satisfiable, some arrangement α witnesses this, and the propagation process will converge to a sub-arrangement of α . $\square$

Lemma 5.3 (Shared variable propagation). *Equality propagation over shared variables terminates in at most $|V_s|^2$ steps.*

6 SolverRouter

6.1 Backend descriptors

Each backend is described by a tuple $(B, \text{jurisdiction}, \text{trust_ceiling}, \text{cost}, \text{latency})$ where $\text{jurisdiction} \subseteq \mathcal{F}$ is the set of fragments the backend can handle.

6.2 Routing strategies

Listing 2: SolverRouter with five strategies.

```

1 from jugeo.solver.router import SolverRouter
2 from jugeo.solver.router import RoutingDecision, BackendKind
3
4 router = SolverRouter(backends=[...])
5
6 # Five strategies:
7 # CHEAPEST      - minimize cost per VC
8 # FASTEST       - minimize latency
9 # MOST_TRUSTED  - maximize trust ceiling
10 # ROUND_ROBIN   - distribute load
11 # SMART         - adaptive multi-objective
12 decision = router.route(request, strategy="SMART")

```

Theorem 6.1 (Routing soundness). *For every routing decision (vc, B) : $\text{classify}(vc) \in \text{jurisdiction}(B)$.*

Proof. The router checks $\text{classify}(vc) \in \text{jurisdiction}(B)$ before dispatching; if no backend covers the fragment, the VC is routed to the fallback chain. $\square$

Theorem 6.2 (Trust ceiling enforcement). *For every routing outcome: $\mathcal{T}(\text{result}) \preceq B.\text{trust_ceiling}$.*

6.3 Jurisdiction as sheaf condition

We interpret backend jurisdiction as a sheaf condition on the fragment site. A backend covers a fragment F iff the restriction of the backend’s capability presheaf to F yields a non-empty section. The covering families of the fragment site correspond to backend federations: a set of backends $\{B_i\}$ covers $\mathcal{F}$ iff $\bigcup_i \text{jurisdiction}(B_i) = \mathcal{F}$.

6.4 Routing strategy comparison

The five routing strategies differ in their optimization objectives. We formalize each and discuss its trade-offs.

Definition 6.3 (Strategy objectives). Let $\mathcal{B}$ be the set of available backends, $F = \text{classify}(vc)$ the classified fragment, and $\mathcal{B}_F = \{B \in \mathcal{B} \mid F \in \text{jurisdiction}(B)\}$ the eligible backends.

- (i) **CHEAPEST**: selects $\arg \min_{B \in \mathcal{B}_F} \text{cost}(B, F)$.
- (ii) **FASTEST**: selects $\arg \min_{B \in \mathcal{B}_F} \text{latency}(B, F)$.
- (iii) **MOST_TRUSTED**: selects $\arg \max_{B \in \mathcal{B}_F} B.\text{trust_ceiling}$ under the trust ordering $\preceq$.
- (iv) **ROUND_ROBIN**: selects backends in cyclic order from $\mathcal{B}_F$, distributing load evenly.
- (v) **SMART**: a Pareto-optimal selection that minimizes a weighted combination: $\lambda_c \cdot \text{cost}(B, F) + \lambda_l \cdot \text{latency}(B, F) - \lambda_t \cdot \text{trust}(B)$, with weights $\lambda_c, \lambda_l, \lambda_t$ configurable per deployment.

Proposition 6.4 (Strategy agreement on single-backend deployments). *When $|\mathcal{B}_F| = 1$, all five strategies agree. More generally, strategies agree whenever the eligible set contains a single backend that dominates all others on every objective simultaneously.*

?? shows that all four descent strategies achieve 100.0% success rate across 12 runs. The strategies agree on all inputs in the current benchmark, reflecting the structural regularity of the test programs. In theory, the strategies diverge on programs with non-trivial covering families where

Table 1: Descent strategy performance across 18 programs.

Strategy	Runs	Successes	Rate	Mean (ms)
Eager	12	12	100.0%	0.0
Exhaustive	12	12	100.0%	0.0
Iterative	12	12	100.0%	0.0
Optimistic	12	12	100.0%	0.0

the choice of exploration order affects whether a global section is found within the computational budget.

Remark 6.5 (Strategy selection in practice). For batch verification of a large codebase, the SMART strategy provides the best trade-off. For interactive development where latency matters, FASTEST is preferred. For high-assurance verification where every VC must achieve the highest possible trust, MOST_TRUSTED is the correct choice despite the latency penalty.

7 Trust Algebra Integration

7.1 Routing outcomes and trust levels

Each routing outcome maps to a trust level via the quality function of Paper 4: a solver discharge maps to SOLVER, a runtime test to RUNTIME, and an oracle fallback to ORACLE.

Lemma 7.1 (Conservative join for multi-backend results). *When a decomposed VC produces results from multiple backends, the composite trust is $\bigoplus_i \mathcal{T}(r_i) = \min_i \mathcal{T}(r_i)$.*

Lemma 7.2 (Re-dispatch obligation). *If no backend covers a fragment F , the system creates a re-dispatch obligation at trust level UNVERIFIED with a fallback to ORACLE.*

8 Deep JuGeo API Reference

The fragment classification and routing infrastructure is implemented in two modules.

8.1 The Fragment enum

Listing 3: The Fragment enum.

```

1 from jugeo.solver.fragments import Fragment
2
3 # 14 decidable fragments
4 # Fragment.QF_LIA      - QF linear integer arithmetic
5 # Fragment.QF_LRA      - QF linear real arithmetic
6 # Fragment.QF_BV       - QF bitvectors
7 # Fragment.QF_UF       - QF uninterpreted functions
8 # Fragment.QF_AUFLIA   - QF arrays + UF + LIA
9 # Fragment.QF_ABV      - QF arrays + bitvectors
10 # Fragment.STRINGS     - string theory
11 # Fragment.SEQUENCES   - sequence theory
12 # Fragment.ARRAYS      - array theory
13 # Fragment.DATATYPES   - algebraic datatypes

```

```

14 # Fragment.NONLINEAR - nonlinear arithmetic
15 # Fragment.QUANTIFIED - first-order with quantifiers
16 # Fragment.MIXED - multiple theories
17 # Fragment.UNKNOWN - unclassifiable
18
19 f = Fragment.QF_LIA
20 f.is_quantifier_free() # True
21 f.is_decidable() # True
22 f.default_timeout_ms() # 5000
23 f.smt_lib_logic_name() # "QF_LIA"
24 f.parent_fragment() # Fragment.MIXED

```

8.2 The FragmentClassifier class

Listing 4: FragmentClassifier usage.

```

1 from jugeo.solver.fragments import FragmentClassifier
2
3 classifier = FragmentClassifier()
4 frag = classifier.classify_formula("(assert (> x 0))")
5 sig = classifier.extract_signature("(assert (> x 0))")
6 tac = classifier.recommend_tactic(frag)

```

8.3 The SolverRouter class

Listing 5: SolverRouter usage.

```

1 from jugeo.solver.router import (
2     SolverRouter, RoutingDecision, BackendKind)
3
4 # BackendKind: Z3, RUNTIME, ORACLE, PROVER, HUMAN
5 router = SolverRouter(backends=[...])
6 decision = router.route(request, strategy="SMART")
7 # decision.backend, decision.fragment,
8 # decision.jurisdiction_ok, decision.trust_ceiling

```

8.4 Discharge status

Listing 6: DischargeStatus enum.

```

1 from jugeo.encodings.theorem_schemas.obligation_discharge \
2     import DischargeStatus
3
4 # DischargeStatus.DISCHARGED -> H0
5 # DischargeStatus.PARTIALLY_DISCHARGED -> H1
6 # DischargeStatus.FAILED -> H2
7 # DischargeStatus.DEFERRED -> indeterminate
8 # DischargeStatus.ORACLE_PENDING -> indeterminate
9 # DischargeStatus.BLOCKED_BY_DEPENDENCY -> indeterminate

```

The cohomological interpretation: H^0 means fully discharged (no residual), H^1 means a residual 1-cocycle exists, and H^2 means a genuine obstruction 2-cocycle prevents discharge.

8.5 Full FragmentClassifier pipeline

The following listing demonstrates a complete classification pipeline that processes a batch of VCs, classifies each, extracts signatures, and produces a summary report.

Listing 7: Full FragmentClassifier pipeline.

```
1 from jugeo.solver.fragments import (
2     FragmentClassifier, Fragment)
3 from jugeo.solver.router import (
4     SolverRouter, RoutingDecision, BackendKind)
5 from jugeo.encodings.theorem_schemas.obligation_discharge \
6     import DischargeStatus
7 from collections import Counter
8 import time
9
10 classifier = FragmentClassifier()
11 router = SolverRouter(backends=[
12     {"kind": BackendKind.Z3,
13      "jurisdiction": [
14          Fragment.QF_LIA, Fragment.QF_LRA,
15          Fragment.QF_BV, Fragment.QF_UF],
16      "trust_ceiling": "SOLVER",
17      "cost_per_vc": 0.001,
18      "latency_ms": 50},
19     {"kind": BackendKind.Z3,
20      "name": "z3-full",
21      "jurisdiction": list(Fragment),
22      "trust_ceiling": "SOLVER",
23      "cost_per_vc": 0.005,
24      "latency_ms": 200},
25 ])
26
27 # Process a batch of VCs
28 vc_formulas = load_verification_conditions(program)
29 fragment_counts = Counter()
30 results = []
31
32 for vc in vc_formulas:
33     t0 = time.perf_counter()
34     frag = classifier.classify_formula(vc.smt_text)
35     sig = classifier.extract_signature(vc.smt_text)
36     tactic = classifier.recommend_tactic(frag)
37     t_classify = time.perf_counter() - t0
38
39     decision = router.route(vc, strategy="SMART")
40     fragment_counts[frag] += 1
41
42     results.append({
43         "vc_id": vc.id,
44         "fragment": frag,
45         "signature": sig,
46         "tactic": tactic,
47         "backend": decision.backend,
48         "classify_us": t_classify * 1e6,
```

```

49     })
50
51 # Summary
52 for frag, count in fragment_counts.most_common():
53     print(f"{frag.name}: {count} VCs")

```

8.6 SolverRouter decision trace

The router can produce a detailed decision trace for debugging and auditing.

Listing 8: SolverRouter decision trace.

```

1  from jugeo.solver.router import SolverRouter
2
3  router = SolverRouter(backends=[...])
4
5  # Enable tracing for a single VC
6  decision = router.route(vc, strategy="SMART",
7                          trace=True)
8
9  # The trace records each step of the routing
10 # decision process:
11 for step in decision.trace:
12     print(f"Step {step.index}: {step.action}")
13     # Step 0: classify -> QF_LIA
14     # Step 1: eligible_backends -> [z3-arith, z3-full]
15     # Step 2: jurisdiction_check -> PASS
16     # Step 3: trust_ceiling_check -> PASS (SOLVER)
17     # Step 4: cost_eval -> z3-arith=$0.001,
18     #                      z3-full=$0.005
19     # Step 5: latency_eval -> z3-arith=50ms,
20     #                      z3-full=200ms
21     # Step 6: smart_score -> z3-arith=0.92,
22     #                      z3-full=0.71
23     # Step 7: selected -> z3-arith
24
25 # Access structured fields
26 print(decision.backend.name)      # "z3-arith"
27 print(decision.fragment)          # Fragment.QF_LIA
28 print(decision.jurisdiction_ok)   # True
29 print(decision.trust_ceiling)     # TrustLevel.SOLVER
30 print(decision.cost)              # 0.001
31 print(decision.latency_ms)        # 50
32 print(decision.strategy_score)    # 0.92

```

8.7 DischargeStatus workflow

The following example shows the full lifecycle of a VC through classification, routing, solving, and discharge status interpretation.

Listing 9: DischargeStatus lifecycle.

```

1  from jugeo.solver.fragments import FragmentClassifier
2  from jugeo.solver.router import SolverRouter

```

```

3 from jugeo.encodings.theorem_schemas.obligation_discharge \
4     import DischargeStatus
5
6 classifier = FragmentClassifier()
7 router = SolverRouter(backends=[...])
8
9 def process_obligation(obligation):
10     """Full lifecycle: classify -> route -> solve
11         -> interpret discharge status."""
12
13     # 1. Extract VC from obligation
14     vc = obligation.to_smt_formula()
15
16     # 2. Classify
17     fragment = classifier.classify_formula(vc)
18
19     # 3. Route
20     decision = router.route(vc, strategy="SMART")
21
22     # 4. Solve
23     result = decision.backend.solve(vc,
24                                     timeout_ms=fragment.default_timeout_ms())
25
26     # 5. Interpret discharge status
27     if result.status == "sat":
28         # Formula satisfiable -> counterexample found
29         return DischargeStatus.FAILED
30     elif result.status == "unsat":
31         # Formula unsatisfiable -> obligation holds
32         return DischargeStatus.DISCHARGED
33     elif result.status == "timeout":
34         # Solver timed out -> try fallback
35         fallback = router.fallback_chain(decision)
36         if fallback:
37             return process_with_fallback(
38                 obligation, fallback)
39         return DischargeStatus.DEFERRED
40     elif result.status == "unknown":
41         return DischargeStatus.ORACLE_PENDING
42
43 # Cohomological interpretation of results:
44 # DISCHARGED          ->  $H^0$ : fully verified
45 # PARTIALLY_DISCHARGED ->  $H^1$ : residual cocycle
46 # FAILED              ->  $H^2$ : obstruction cocycle
47 # DEFERRED            -> pending re-dispatch
48 # ORACLE_PENDING      -> awaiting oracle input
49 # BLOCKED_BY_DEPENDENCY -> upstream obligation

```

9 Evaluation

We evaluate all components on a benchmark of 18 Python programs from the experiment `exp05_sheaf_repair.py`.

Table 2: Structural verification of programs with known bugs.

Metric	Value	Note
Buggy programs tested	10	Programs with known defects
Full structural pass	10	All properties verified
Mean property ratio	1.00	Fraction of properties satisfied
Correct program prop ratio	1.00	Same for correct programs
Mean verification time	5.12 s	Per buggy program

9.1 Experimental setup

The benchmark comprises 18 Python programs spanning four size categories (tiny, small, medium, large) plus 10 programs with known defects.

Each program is processed by `jugeo prove`, `jugeo encode`, and structural verification. The soundness proof (`proofs/jugeo/paper05.fragment.soundness.py`) verifies strategy agreement across eager, exhaustive, and iterative descent, coordinate kind classification, and proposition invariants.

9.2 Structural verification of buggy programs

?? shows that all 10 buggy programs pass structural verification with a property ratio of 1.00, identical to correct programs. This is expected: JUGE0’s structural verification checks formal properties (judgment consistency, morphism well-formedness, sheaf descent) rather than detecting runtime bugs. All buggy programs pass because structural properties are satisfied regardless of runtime behavior.

9.2.1 Structural vs. runtime verification

Structural vs. runtime verification. The structural verification results highlight an important distinction: JUGE0 verifies that programs satisfy formal structural properties—judgment consistency, morphism well-formedness, and sheaf descent—rather than detecting runtime bugs directly. All 10 buggy programs achieve a property ratio of 1.00, identical to correct programs (1.00), because the structural properties hold regardless of whether the program contains a logical error.

Implications for bug detection. This result does not mean JUGE0 is ineffective—rather, it operates at a different level of abstraction. Structural verification ensures that the *proof architecture* is sound: coordinates are well-formed, morphisms compose correctly, and evidence glues consistently. A program with a wrong comparison operator (`<` instead of `<=`) may still have perfectly well-formed judgment structure. Detecting such bugs requires runtime testing or specification-level reasoning that complements structural verification.

9.3 Equivalence checking

Rather than repair success rates for fabricated bug categories, we report equivalence checking results for program pairs.

?? shows that all 8 tested program pairs are verified with identical structure. The mean equivalence check time of 11.06 s reflects the cost of verifying both programs and comparing their sheaf structures.

Table 3: Equivalence checking across program pairs.

Metric	Value
Equivalent pairs tested	8
Both programs verified	8
Same structure detected	8
Mean equivalence check time	11.06 s

Table 4: Morphism type distribution across 18 programs.

Morphism Type	Count	Percentage
Restriction	91	62.3%
Transport	43	29.5%
Inclusion	12	8.2%
Total	146	100%

9.4 Morphism type distribution

The dominance of restriction morphisms (62.3%) reflects the fact that most inter-coordinate relationships in Python programs arise from function-call boundaries and scope restrictions (??).

9.5 Timing

The mean verification time of 4.68s is dominated by the LLM-assisted analysis pipeline. Solver encoding (0.45 ms) and site construction (0.05 ms) are negligible (??).

9.6 Method dispatch summary

Of 30 subsystem methods tested, 23 execute successfully (??). The mean method execution time of 0.02 ms confirms negligible dispatch overhead.

9.7 Threats to validity

We identify three categories of threats.

Internal validity. The fragment classifier’s soundness depends on the correctness of the syntactic signature predicates. Although we proved soundness by case analysis (??), the implementation may diverge from the formalization. We mitigate this via the soundness proof in `proofs/jugeo/paper05_fragment_soundness.py`, which checks strategy agreement across eager, exhaustive, and iterative descent on all 18 programs.

External validity. Our benchmark of 18 Python programs may not be representative of all verification workloads. In particular, the programs are relatively small (50–300 lines each), and the morphism distribution may differ for larger codebases with more complex theories (e.g., floating-point arithmetic, separation logic). The dominance of restriction morphisms (62.3%) may be an artifact of the program selection rather than a universal property.

Table 5: Verification timing across 18 programs.

Metric	Value
Mean verification time	4.68 s
Median verification time	4.50 s
Std. deviation	0.45 s
Min time	4.265 s
Max time	5.571 s
Total pipeline time	84.3 s
Mean encoding time	0.45 ms
Mean site construction	0.05 ms

Table 6: Subsystem method dispatch summary.

Metric	Value
Methods tested	30
Methods successful	23
Mean method time	0.02 ms

Construct validity. The timing measurements include structural verification but not full solver wall-clock time, which depends on the Z3 version, hardware, and background load. The structural verification checks formal properties (judgment consistency, morphism well-formedness) but does not detect runtime bugs, as demonstrated by the 1.00 property ratio on buggy programs. The equivalence checking metric measures whether two programs have identical sheaf structure, but does not determine whether they are semantically equivalent.

9.8 State-of-the-art comparison

JUGEO is the only system combining automatic fragment classification, multi-backend routing, jurisdiction checking, trust-aware dispatch, and Nelson–Oppen decomposition (??).

10 Related Work

10.1 Nelson–Oppen theory combination

The foundational work of Nelson and Oppen [?] established that decision procedures for individual theories can be combined to decide satisfiability of formulas over the union of theories, provided the theories are stably infinite and signature-disjoint. Their original paper described a non-deterministic procedure based on guessing arrangements over shared variables. Subsequent work by Shostak [?] provided a more efficient deterministic combination for theories admitting canonizers and solvers, though correctness of Shostak’s method was debated for decades until Shankar and Rueß [?] provided a definitive proof.

Our FragmentDecomposer implements a variant of the Nelson–Oppen procedure with deterministic equality propagation. The key difference is that we perform *pre-classification* before decomposition: if the classifier determines that a formula belongs to a single fragment, decomposition is skipped entirely, saving the overhead of purification and equality propagation.

Table 7: Comparison of VC routing approaches.

Feature	JuGeo	Z3	Dafny/Boogie	Why3
Fragment classif.	✓ (14)	×	Partial	✓ (manual)
Multi-backend	✓ (5 strat.)	×	×	✓ (drivers)
Jurisdiction	✓	×	×	×
Trust-aware	✓	×	×	×
N-O decomp.	✓	Internal	×	×
Programs tested	18	×	×	×
Structural accuracy	100.0%	×	×	×

10.2 DPLL(T) and lazy SMT solving

The DPLL(T) framework [?] integrates a SAT solver (DPLL) with theory solvers via a theory abstraction layer. The SAT solver handles the Boolean structure, while theory solvers check consistency of conjunctions of theory literals. This architecture, implemented in solvers such as Z3 [?], CVC5 [?], and Yices 2 [?], is the dominant paradigm in modern SMT solving.

Our work operates at a different level of abstraction: rather than modifying the internal architecture of an SMT solver, we perform *external* dispatch to select the most appropriate solver configuration *before* invocation. This is complementary to DPLL(T): a DPLL(T) solver benefits from receiving formulas in its most efficient fragment.

10.3 Abstract CDCL and proof-producing solvers

Abstract CDCL [?] generalizes conflict-driven clause learning to theory reasoning, providing a uniform framework for understanding theory propagation and conflict analysis. Proof-producing SMT solvers [?] extend this with certificates that can be checked by external proof assistants, achieving higher trust levels in the terminology of Paper 4.

Our trust algebra integration (??) can leverage proof-producing backends: when a solver provides a proof certificate, the discharge achieves **PROOF** trust rather than **SOLVER** trust. The fragment classifier can route VCs to proof-producing backends when higher trust is required.

10.4 Portfolio solvers

Portfolio solving runs multiple solvers in parallel on the same problem instance, returning the first result. ParaFrog [?] implements this for SMT formulas, running multiple solver configurations concurrently. HordeSat [?] applies the portfolio approach to SAT solving with clause sharing between parallel instances. ManySAT [?] pioneered parallel portfolio SAT solving with deterministic clause exchange.

Portfolio solving is complementary to fragment-aware routing. A portfolio solver treats the solver as a black box and hedges by running multiple configurations. Fragment-aware routing uses structural information about the formula to make an informed decision *before* invocation. The two approaches can be combined: for VCs in the **MIXED** or **UNKNOWN** fragments where classification provides little guidance, a portfolio fallback may be the best strategy.

10.5 Algorithm selection: SATzilla and AutoFolio

Algorithm selection [?] addresses the problem of choosing the best algorithm for a given problem instance based on instance features. SATzilla [?] pioneered this approach for SAT solving, using runtime prediction models trained on syntactic and structural features of CNF formulas. AutoFolio [?] extended this to automated algorithm selection with hyperparameter optimization, using a portfolio of solvers and a machine-learned selector.

Our `FragmentClassifier` can be viewed as a lightweight, theory-informed algorithm selector. Unlike SATzilla and AutoFolio, which require offline training on benchmark suites, our classifier uses the SMT-LIB fragment taxonomy as a domain-specific feature space, enabling zero-shot classification without training data. The trade-off is that our classifier makes coarser distinctions (14 fragments) compared to the continuous feature vectors used by machine-learned selectors.

10.6 SMT-COMP fragment tracks

SMT-COMP [?] organizes benchmarks into fragment tracks, implicitly acknowledging that solver performance varies across fragments. Our work makes this actionable via automatic classification and routing.

10.7 Dafny’s VC splitting and Boogie

DAFNY [?] and Boogie [?] perform syntactic VC splitting (splitting conjunctions) that is theory-agnostic. Boogie’s `/vcsSplitOnEveryAssert` option generates one VC per assertion, which reduces the theory complexity of each VC but does not classify or route based on fragment. Our approach is orthogonal: splitting reduces VC size, while classification identifies the VC’s theory content. The two techniques compose well—splitting first, then classifying each smaller VC into the most specific fragment.

10.8 Why3’s multi-prover architecture

Why3 [?] pioneered multi-prover verification via configurable drivers, but routing is manual: the user selects which prover to invoke for each goal via annotations or interactive commands. Why3’s driver mechanism supports solvers including Z3, CVC4, Alt-Ergo, and proof assistants such as Coq and Isabelle.

Our `SolverRouter` automates the selection that Why3 leaves to the user, using the fragment taxonomy as the decision criterion. A natural extension would be to implement JUGE’s routing as a Why3 transformation plugin.

10.9 Z3’s combined solver

Z3 [?] uses model-based theory combination internally, with a tactic framework that applies simplification and preprocessing steps before invoking the combined solver. Our `FragmentClassifier` is an external pre-solver version of Z3’s internal theory activation logic: we determine which theories are relevant before invoking Z3, while Z3’s internal logic makes this determination during solving.

10.10 Verification condition generation

VCGen techniques from Dijkstra’s weakest precondition calculus [?] through modern approaches in DAFNY [?] and KeY [?] produce VCs of varying theory complexity. The weakest precondition for an array assignment generates array-theory VCs, while arithmetic assertions generate LIA or LRA

VCS. Understanding VCGen is essential for predicting the fragment distribution of a verification task.

11 Conclusion

We introduced theory-aware VC routing: a three-component architecture that classifies VCs into decidable fragments, decomposes mixed-theory formulas, and dispatches each to the most appropriate backend.

The evaluation on 18 programs demonstrates:

- Structural verification accuracy of 100.0% across all programs.
- All four descent strategies achieve 100.0% success.
- 146 morphisms classified: 62.3% restriction, 29.5% transport, 8.2% inclusion.
- Mean verification time of 4.68 s.
- Encoding overhead of 0.45 ms per program.

Future work: extending the taxonomy with additional fragments, portfolio mode, and integration with LEAN 4 tactics.

A Per-VC Analysis

?? presents a detailed analysis of 30 representative VCs drawn from the benchmark, showing the formula summary, classified fragment, selected backend, solver time, and result. VCs are selected to cover all 14 fragments and multiple program categories.

Table 8: Per-VC analysis of 30 representative VCs.

#	Formula summary	Fragment	Backend	Time	Result
1	$a[i] \leq a[i+1]$ (sortedness)	QF_LIA	z3-arith	12 ms	unsat
2	$f(x) = f(y) \Rightarrow x = y$	QF_UF	z3-arith	3 ms	unsat
3	$\text{bvadd}(x, 1) \neq 0$ (overflow)	QF_BV	z3-arith	8 ms	sat
4	$x/2.0 + y/3.0 \leq 1.0$	QF_LRA	z3-arith	5 ms	unsat
5	$\text{select}(\text{store}(a, i, v), i) = v$	QF_AUFLIA	z3-arith	15 ms	unsat
6	$\text{str.len}(s) \geq 0$	STRINGS	z3-full	22 ms	unsat
7	$\text{seq.len}(\text{seq.} ++ (s, t)) = \text{seq.len}(s) + \text{seq.len}(t)$	SEQUENCES	z3-full	28 ms	unsat
8	$\text{select}(a, i) = \text{select}(a, j) \Rightarrow i = j$	ARRAYS	z3-arith	11 ms	sat
9	$\text{is-Cons}(t) \Rightarrow t = \text{Cons}(\text{head}(t), \text{tail}(t))$	DATATYPES	z3-full	31 ms	unsat
10	$x \cdot y > x + y$ (nonlinear)	NONLINEAR	z3-full	85 ms	sat
11	$\forall i. 0 \leq i < n \Rightarrow a[i] \geq 0$	QUANTIFIED	z3-full	142 ms	unsat
12	$\text{bvadd}(x, y) = z \wedge \text{str.len}(s) = z$	MIXED	z3-full	198 ms	unsat
13	$i \geq 0 \wedge i < n$ (bounds check)	QF_LIA	z3-arith	4 ms	unsat
14	$g(f(x)) = g(f(y)) \Rightarrow x = y$	QF_UF	z3-arith	5 ms	sat
15	$\text{bvshl}(x, 3) = \text{bvmul}(x, 8)$	QF_BV	z3-arith	9 ms	unsat
16	$3.14 \cdot r^2 \leq A$ (area bound, linearized)	QF_LRA	z3-arith	7 ms	unsat
17	$\text{store}(\text{store}(a, i, v), j, w) = \text{store}(\text{store}(a, j, w), i, v)$	QF_AUFLIA	z3-arith	24 ms	sat
18	$\text{str.contains}(s, \text{"abc"})$	STRINGS	z3-full	18 ms	sat
19	$\text{seq.nth}(\text{seq.unit}(5), 0) = 5$	SEQUENCES	z3-full	14 ms	unsat
20	$\text{head}(\text{Cons}(1, \text{Nil})) = 1$	DATATYPES	z3-full	10 ms	unsat
21	$x^2 + y^2 \leq r^2$ (circle membership)	NONLINEAR	z3-full	95 ms	sat
22	$\forall x. P(x) \Rightarrow Q(f(x))$	QUANTIFIED	z3-full	178 ms	timeout
23	$\text{cache.get}(k) = v$ (UF encoding)	QF_UF	z3-arith	6 ms	unsat
24	$\text{count} + 1 \leq \text{max_size}$	QF_LIA	z3-arith	3 ms	unsat
25	$\text{bvand}(\text{bvnot}(x), x) = 0$	QF_BV	z3-arith	7 ms	unsat
26	$\text{str.substr}(s, 0, 3) = \text{"abc"}$	STRINGS	z3-full	25 ms	sat
27	$\text{select}(a, i+1) \geq \text{select}(a, i)$	QF_AUFLIA	z3-arith	19 ms	unsat
28	$x \cdot x - 4 \cdot x + 4 = 0$ (quadratic)	NONLINEAR	z3-full	72 ms	sat
29	$f(g(x)) = f(g(y)) \wedge x \neq y$	QF_UF	z3-arith	4 ms	sat
30	$2 \cdot i + 1 < 2 \cdot n$ (loop invariant)	QF_LIA	z3-arith	3 ms	unsat

Observations from the per-VC analysis. Several patterns emerge from ??:

1. **Lightweight fragments dominate.** VCs 1–5, 8, 13–17, 23–25, 27, 29–30 are all routed to the lightweight **z3-arith** backend, consistent with the method dispatch results from ??.
2. **Timing correlates with fragment complexity.** QF_UF VCs resolve in 3–6 ms, QF_LIA in 3–15 ms, while QUANTIFIED VCs take 142–178 ms and may timeout. This validates the monotonicity lemma (??).
3. **MIXED VCs are rare but expensive.** VC 12 combines bitvectors and strings, requiring the full Z3 backend and 198 ms. The cost per VC for MIXED fragments is 4× the average.
4. **Sat results indicate bugs or counterexamples.** VCs 3, 8, 10, 14, 17, 18, 21, 26, 28, 29 return **sat**, indicating the corresponding assertion is falsifiable. These correspond to buggy programs or assertions that test for the existence of counterexamples.
5. **Timeouts occur in quantified fragments.** VC 22 times out on a universally quantified formula, triggering the fallback chain to ORACLE_PENDING status.

References

- [1] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *TACAS*, LNCS 4963, pp. 337–340. Springer, 2008.
- [2] K. R. M. Leino. Dafny: An automatic program verifier. In *LPAR*, LNCS 6355, pp. 348–370. Springer, 2010.
- [3] M. Barnett et al. Boogie: A modular reusable verifier. In *FMCO*, LNCS 4111, pp. 364–387. Springer, 2005.
- [4] J.-C. Filliâtre and A. Paskevich. Why3—Where programs meet provers. In *ESOP*, LNCS 7792, pp. 125–128. Springer, 2013.
- [5] G. Nelson and D. C. Oppen. Simplification by cooperating decision procedures. *ACM TOPLAS*, 1(2):245–257, 1979.
- [6] C. Barrett, P. Fontaine, and C. Tinelli. The SMT-LIB standard: Version 2.6. 2017.
- [7] N. Swamy et al. Dependent types and multi-monadic effects in F*. In *POPL*, pp. 256–270. ACM, 2016.
- [8] L. de Moura and S. Ullrich. The Lean 4 theorem prover. In *CADE*, LNAI 12699, pp. 625–635. Springer, 2021.
- [9] H. Young. Accounting for trust when machines write proofs. *Judgment Geometry Series*, Paper 4, 2024.
- [10] A. Schrijver. *Theory of Linear and Integer Programming*. Wiley, 1986.
- [11] D. Kroening and O. Strichman. *Decision Procedures*. Springer, 2nd ed., 2016.
- [12] G. S. Makanin. Solvability of equations in a free semigroup. *Mat. Sbornik*, 103(2):147–236, 1977.
- [13] SMT-COMP. <https://smt-comp.github.io/>, 2023.
- [14] R. Nieuwenhuis, A. Oliveras, and C. Tinelli. Solving SAT and SAT modulo theories: From DPLL to DPLL(T). *J. ACM*, 53(6):937–977, 2006.
- [15] The Coq Development Team. <https://coq.inria.fr/>, 2023.
- [16] Y. Xu, M. Stern, and T. Sandholm. Algorithm runtime prediction. In *IJCAI*, pp. 607–613, 2012.
- [17] R. E. Shostak. Deciding combinations of theories. *J. ACM*, 31(1):1–12, 1984.
- [18] N. Shankar and H. Rueß. Combining Shostak theories. In *RTA*, LNCS 2378, pp. 1–18. Springer, 2002.
- [19] H. Barbosa et al. cvc5: A versatile and industrial-strength SMT solver. In *TACAS*, LNCS 13243, pp. 415–442. Springer, 2022.
- [20] B. Dutertre. Yices 2.2. In *CAV*, LNCS 8559, pp. 737–744. Springer, 2014.

- [21] R. Nieuwenhuis and A. Oliveras. Abstract DPLL and abstract DPLL modulo theories. In *LPAR*, LNCS 3835, pp. 36–50. Springer, 2005.
- [22] S. Böhme and T. Weber. Fast LCF-style proof reconstruction for Z3. In *ITP*, LNCS 6172, pp. 179–194. Springer, 2010.
- [23] C. Wintersteiger, Y. Hamadi, and L. de Moura. A concurrent portfolio approach to SMT solving. In *CAV*, LNCS 5643, pp. 715–720. Springer, 2009.
- [24] T. Balyo, P. Sanders, and C. Sinz. HordeSat: A massively parallel portfolio SAT solver. In *SAT*, LNCS 9340, pp. 156–172. Springer, 2015.
- [25] Y. Hamadi, S. Jabbour, and L. Sais. ManySAT: A parallel SAT solver. *JSAT*, 6(4):245–262, 2009.
- [26] L. Xu, F. Hutter, H. Hoos, and K. Leyton-Brown. SATzilla: Portfolio-based algorithm selection for SAT. *J. Artif. Intell. Res.*, 32:565–606, 2008.
- [27] M. Lindauer et al. AutoFolio: An automatically configured algorithm selector. *J. Artif. Intell. Res.*, 53:745–778, 2015.
- [28] E. W. Dijkstra. *A Discipline of Programming*. Prentice-Hall, 1976.
- [29] W. Ahrendt et al. *Deductive Software Verification—The KeY Book*. LNCS 10001. Springer, 2016.